Independent University, Bangladesh

CSE313 Compiler Construction · Practice Set A · Mid + Quiz · Time: 1h30m

New questions in the Aut-23 shape. Solve on paper first, then check.

Question paper

Mid Question (Total marks: 20)

1. Prove whether the following grammar is Ambiguous or NOT. If the grammar is ambiguous, convert it to unambiguous form. **[7]**

```
B → B or B | B and B | not B | ( B ) | id
```

2. Write the First() and Follow() of the following Grammar. Draw LL(1) Parsing Table and parse the string `[ ] d a b c b` showing all steps. If parsing is not possible, point out the reasons. (b) Parse `a c b b`. V = {E, F, G, H}, T = {a, b, c, d, [,], ε}, S = {E} **[8]**

```
E → a F G | H E
F → b | ε
G → c F | ε
H → d | [ F ]
```

3. Find True/False: **[5]**

(a) Every LL(1) grammar is unambiguous.
(b) The FOLLOW set of a non-terminal can contain ε.
(c) An LL(1) parsing table has one row for each terminal.
(d) Syntax analysis comes after lexical analysis.
(e) A Bottom-Up Parser builds the parse tree from the root down to the leaves.

Quiz Question (Total marks: 10)

1. Convert the following Non-Deterministic Grammar (NDG) to Deterministic Grammar (DG): **[5]**

```
S → i E t S | i E t S e S | a
E → b c | b d | b
```

2. Convert the following Left Recursive Grammar (LRG) to Right Recursive Grammar (RRG): **[5]**

```
S → S a B | S b | c
B → B d | e
```

Answers (exam layout, with comments)

Answer 1 [7]

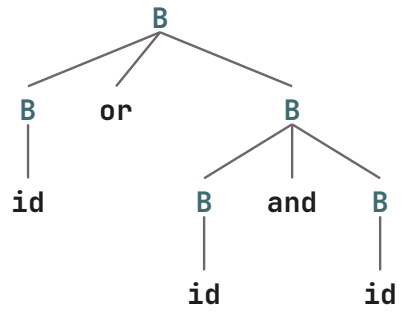
WRITE THIS

Yes, the grammar is ambiguous. String: `id or id and id`

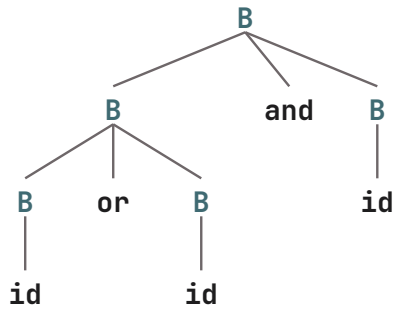
LMD 1 (or at the root): B ⇒ B or B ⇒ id or B ⇒ id or B and B ⇒ id or id and B ⇒ id or id and id

LMD 2 (and at the root): B ⇒ B and B ⇒ B or B and B ⇒ id or B and B ⇒ id or id and B ⇒ id or id and id

Tree 1



Tree 2



Since for the string `id or id and id` there exists more than one leftmost derivation parse tree, the grammar is ambiguous.

[Assuming not > and > or in precedence; or and and are left associative; not is a prefix operator (right associative)]

Operator	Precedence	Associativity
not	highest	right (prefix)
and	middle	left
or	lowest	left

Unambiguous grammar:

```
B → B or T | T
T → T and F | F
F → not F | ( B ) | id
```

Checked by computer: same strings as the original (all 207 strings up to length 7), and each has exactly one LMD.

> **Lose marks by:** forgetting "not F" must recurse on the right (not not id); dropping (B).

Answer 2 [8]

WRITE THIS

Numbered rules: 1. E → a F G 2. E → H E 3. F → b 4. F → ε 5. G → c F 6. G → ε 7. H → d 8. H → [F]

Step 1 · FIRST & FOLLOW

Non-terminal	FIRST	FOLLOW
E	{ a, d, [}	{ $ }
F	{ b, ε }	{ c, $,] }
G	{ c, ε }	{ $ }
H	{ d, [}	{ a, d, [}

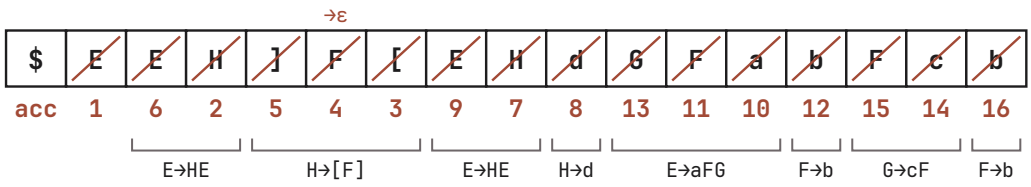
Step 2 · LL(1) parsing table

	a	b	c	d	[	]	$
E	E→aFG			E→HE	E→HE		
F		F→b	F→ε			F→ε	F→ε
G			G→cF				G→ε
H				H→d	H→[F]		

No cell has more than one rule ∴ **the grammar is LL(1)**.

Step 3a · Parse `[ ] d a b c b` (horizontal stack)

Buffer: `[ ] d a b c b $` LA ↑ starts at `[`



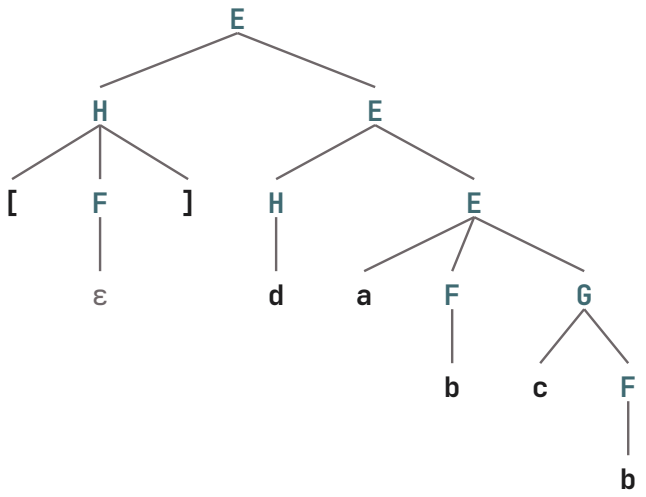
How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

1. pop E (LA = [): E→HE, push E H
2. pop H (LA = [): H→[F], push] F [
3. match [, pop it, move LA →]
4. pop F (LA =]): F→ε, push nothing
5. match], pop it, move LA → d
6. pop E (LA = d): E→HE, push E H
7. pop H (LA = d): H→d, push d
8. match d, pop it, move LA → a
9. pop E (LA = a): E→aFG, push G F a
10. match a, pop it, move LA → b
11. pop F (LA = b): F→b, push b
12. match b, pop it, move LA → c
13. pop G (LA = c): G→cF, push F c
14. match c, pop it, move LA → b
15. pop F (LA = b): F→b, push b
16. match b, pop it, move LA → $

acc. Top = $ and LA = $ → accept

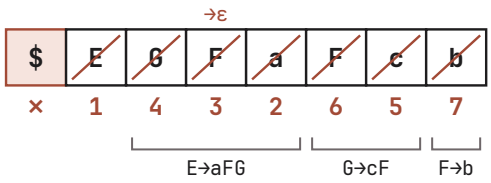
Parse tree



Leaves left to right (skip ε): `[ ] d a b c b` = the input. Stack top $ and LA $ ∴ the string is accepted, it is syntactically correct.

Step 3b · Parse `a c b b` (horizontal stack)

Buffer: `a c b b $` LA ↑ starts at `a`



How to read it: red number under a cell = the order it was popped. Bracket = what that pop pushed (right side, reversed). →ε above a cell = it was replaced by ε (push nothing). × = where parsing stopped.

Step labels (write these under the cells on paper):

1. pop E (LA = a): E→aFG, push G F a
2. match a, pop it, move LA → c
3. pop F (LA = c): F→ε, push nothing
4. pop G (LA = c): G→cF, push F c
5. match c, pop it, move LA → b
6. pop F (LA = b): F→b, push b
7. match b, pop it, move LA → b

err. Top = $ but LA = b (input is left over) → error

Parsing is not possible. Reason: Top = $ but LA = b (input is left over) → error. ∴ error, the string `a c b b` is not in the language.

▼ How the FIRST / FOLLOW were found (rough work, not needed on paper)

- FIRST(E): E → a F G gives { a }. E → H E gives { d, [}. Total { a, d, [}.
- FIRST(F): F → b gives { b }. F → ε gives ε. Total { b, ε }.
- FIRST(G): G → c F gives { c }. G → ε gives ε. Total { c, ε }.
- FIRST(H): H → d gives { d }. H → [F] gives { [}. Total { d, [}.
- FOLLOW(E): start symbol → add $. Total { $ }.
- FOLLOW(F): in E → a F G: after F is G → add { c }; G can be ε → add FOLLOW(E). in G → c F: nothing after F → add FOLLOW(G). in H → [F]: after F is] → add {] }. Total { c, $,] }.

- FOLLOW(G): in E → a F G: nothing after G → add FOLLOW(E). Total { $ }.
- FOLLOW(H): in E → H E: after H is E → add { a, d, [}. Total { a, d, [}.

> **Lose marks by:** FOLLOW(F) has three sources: c (from a F G), FOLLOW(G) = { $ }, and] (from [F]).

Answer 3 [5]

WRITE THIS

#	Answer	Reason (one line)
(a)	**True**	Each cell has at most one rule, so each string has only one derivation.
(b)	**False**	FOLLOW holds terminals and $ only; ε never goes in FOLLOW.
(c)	**False**	One row per non-terminal; one column per terminal plus $.
(d)	**True**	The parser reads the tokens that the lexical analyzer makes.
(e)	**False**	BUP goes from the leaves (input) up to the root (start symbol). TDP goes root down.

Quiz Answer 1 [5]

WRITE THIS

Round 1: in S, `i E t S`, `i E t S e S` all start with `i E t S`. Take it out once:

```
S → i E t S S' | a
S' → ε | e S
```

Round 2: in E, `b c`, `b d`, `b` all start with `b`. Take it out once:

```
E → b E'
E' → c | d | ε
```

Final DG:

```
S → i E t S S' | a
S' → ε | e S
E → b E'
E' → c | d | ε
```

Quiz Answer 2 [5]

WRITE THIS

S → S a B | S b | c: α = `a B`, `b`, β = `c`.

```
S → c S'
S' → a B S' | b S' | ε
```

B → B d | e: α = `d`, β = `e`.

```
B → e B'
B' → d B' | ε
```

Final RRG:

```
S → c S'
S' → a B S' | b S' | ε
B → e B'
B' → d B' | ε
```

Rule: A → Aα | β ⇒ A → βA', A' → αA' | ε. "β goes first, α goes to the new guy, new guy ends with ε."